DCU Phone-Home Capability

Developer Specification

iMatrix Systems

March 2026

Contents

DCU Phone-Home Capability — Developer Specification **2**

Table of Contents . . . 2
1. Document Purpose . . . 3
2. Glossary . . . 3
3. System Architecture . . . 5
    3.1 High-Level Topology . . . 5
    3.2 Trust Boundaries . . . 6
    3.3 Component Ownership . . . 6
4. Security Model and Key Management . . . 6
    4.1 Cryptographic Assets . . . 6
    4.2 Key Generation Policy . . . 7
    4.3 Bastion Host Key Pinning . . . 7
    4.4 HMAC Shared Secret (FC-1 DCU) . . . 7
5. Component Specifications . . . 8
    5.1 DCU: Key Generation and Storage . . . 8
    5.2 DCU: Bastion Registration . . . 9
    5.3 DCU: Phone-Home SSH Client Script . . . 11
    5.4 DCU: DoIP Phone-Home Service Handler . . . 12
    5.5 FC-1: Signal Relay and DCU Provisioning . . . 15
    5.6 Bastion: Registration API . . . 16
    5.7 Bastion: Signal Dispatcher . . . 18
    5.8 Backend/CoAP Server: Operator Trigger API . . . 20
6. DoIP Integration . . . 21
    6.1 Custom Routine Control Service . . . 21
    6.2 Integration into Existing DoIP Server . . . 21
    6.3 Replay Protection and HMAC . . . 25
7. Provisioning Flow . . . 25
    7.1 Zero-Touch Provisioning Sequence . . . 26
    7.2 Provisioning Details . . . 27
    7.3 Provisioning State Machine . . . 27
    7.4 Re-Provisioning Behavior . . . 29
    7.4 Prerequisites . . . 29
8. Operational Phone-Home Flow . . . 29

9. Configuration Reference . . . 30
9.1 DCU Configuration File: `/etc/phonehome/phonehome.conf` . . . 30
9.2 FC-1 Configuration . . . 30
9.3 Bastion SSH Server Configuration (`/etc/ssh/sshd_config` additions) . . . 31
10. Error Handling and Edge Cases . . . 31
11. Testing Procedures . . . 32
11.1 Unit Tests . . . 32
11.2 Integration Tests . . . 33
11.3 End-to-End Tests . . . 34
11.4 Security Tests . . . 34
11.5 Fleet Regression Test . . . 35
12. Recommended Source Files and Dependencies . . . 36
12.1 DoIP Client Utility (for FC-1 → DCU signaling) . . . 36
12.2 OpenSSH (BusyBox Dropbear alternative) . . . 37
12.3 OpenSSL / LibreSSL . . . 38
12.4 Key Source Files to Create or Obtain . . . 38
12.5 Existing Codebase Review Checklist . . . 38
13. Open Issues and Decision Log . . . 38

DCU Phone-Home Capability — Developer Specification

Version: 2.0.0 **Status:** Implementation Complete (FC-1 + DCU provisioning validated 2026-03-17) **Applies To:** FC-1 (BusyBox 4G Telematics Gateway), DCU (Diagnostic Control Unit, Ubuntu/BusyBox embedded), Bastion Server **Standards:** ISO 13400 (DoIP), ISO 21434, UNECE WP.29, RFC 4253 (SSH)

Table of Contents

1. Document Purpose
2. Glossary
3. System Architecture
4. Security Model and Key Management
5. Component Specifications
   - 5.1 DCU: Key Generation and Storage
   - 5.2 DCU: Bastion Registration
   - 5.3 DCU: Phone-Home SSH Client Script
   - 5.4 DCU: DoIP Phone-Home Service Handler
   - 5.5 FC-1: Signal Relay Agent
   - 5.6 Bastion: Registration API
   - 5.7 Bastion: Signal Dispatcher
   - 5.8 Backend/CoAP Server: Operator Trigger API
6. DoIP Integration
   - 6.1 Custom Routine Control Service
   - 6.2 Integration into Existing DoIP Server
   - 6.3 Replay Protection and HMAC
7. Provisioning Flow

8. Operational Phone-Home Flow
9. Configuration Reference
10. Error Handling and Edge Cases
11. Testing Procedures
    - 11.1 Unit Tests
    - 11.2 Integration Tests
    - 11.3 End-to-End Tests
    - 11.4 Security Tests
    - 11.5 Fleet Regression Test
12. Recommended Source Files and Dependencies
13. Open Issues and Decision Log

---

1. Document Purpose

This specification defines all software components, interfaces, data formats, and test procedures required to implement a **phone-home capability** for a Diagnostic Control Unit (DCU) located behind the FC-1 4G telematics gateway.

The phone-home capability allows an operator, via a web UI, to establish an on-demand reverse SSH session to a DCU that has no publicly routable IP address. The design satisfies:

- **ISO 21434** cryptographic key confidentiality (private keys never leave the originating device)
- **UNECE WP.29** cybersecurity management system requirements for remote access
- Operational requirements for a large fleet (100s–1000s of units)

This document is intended for developers implementing the solution. It assumes familiarity with SSH, DoIP (ISO 13400), Linux shell scripting, and embedded systems programming.

---

2. Glossary

Term	Definition
DCU	Diagnostic Control Unit. Vehicle ECU/gateway running Ubuntu or BusyBox in an embedded environment. Runs a DoIP server.
FC-1	Field Controller 1. BusyBox-based 4G telematics gateway. Acts as NAT gateway for the DCU. Already has its own phone-home / reverse SSH tunnel to the Bastion.
Bastion	The cloud-hosted SSH bastion server. Accepts inbound reverse SSH tunnels from both FC-1 and DCU. Provides operator access to tunneled devices.
DoIP	Diagnostics over IP (ISO 13400-2). The transport and application protocol used by the DCU's diagnostic server.

Term	Definition
UDS	Unified Diagnostic Services (ISO 14229). Diagnostic service layer used over DoIP.
SID	Service Identifier. One-byte code identifying a UDS service.
RoutineControl	UDS service 0x31. Used here to carry the phone-home trigger command.
routineIdentifier	Two-byte sub-identifier within RoutineControl. 0xF0A0 is reserved for this phone-home function.
Reverse SSH Tunnel	An SSH connection initiated by the device (outbound) that makes a port on the Bastion forward inbound connections back to the device.
HMAC	Hash-based Message Authentication Code. Used to authenticate DoIP phone-home trigger messages.
ED25519	Elliptic curve SSH key algorithm. Preferred for embedded environments: compact keys, fast operations, strong security.
Provisioning Token	Short-lived, scoped bearer token used for one-time device registration with the Bastion.
Serial Number	Unique device identifier. Both FC-1 and DCU have independent serial numbers.
CoAP	Constrained Application Protocol. Used by the backend server for lightweight IoT signaling.

3. System Architecture

3.1 High-Level Topology

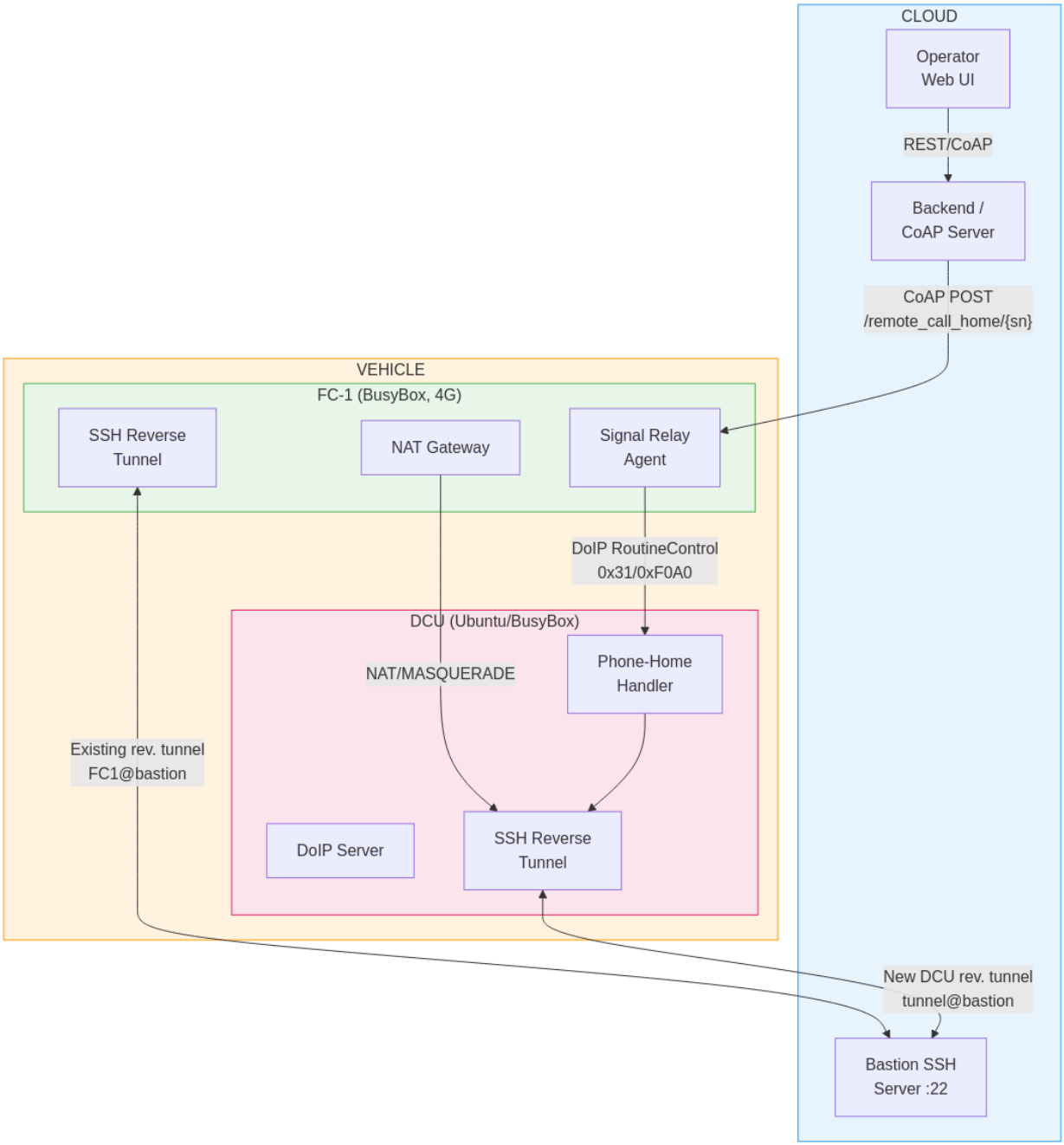

Figure 1: System Architecture

3.2 Trust Boundaries

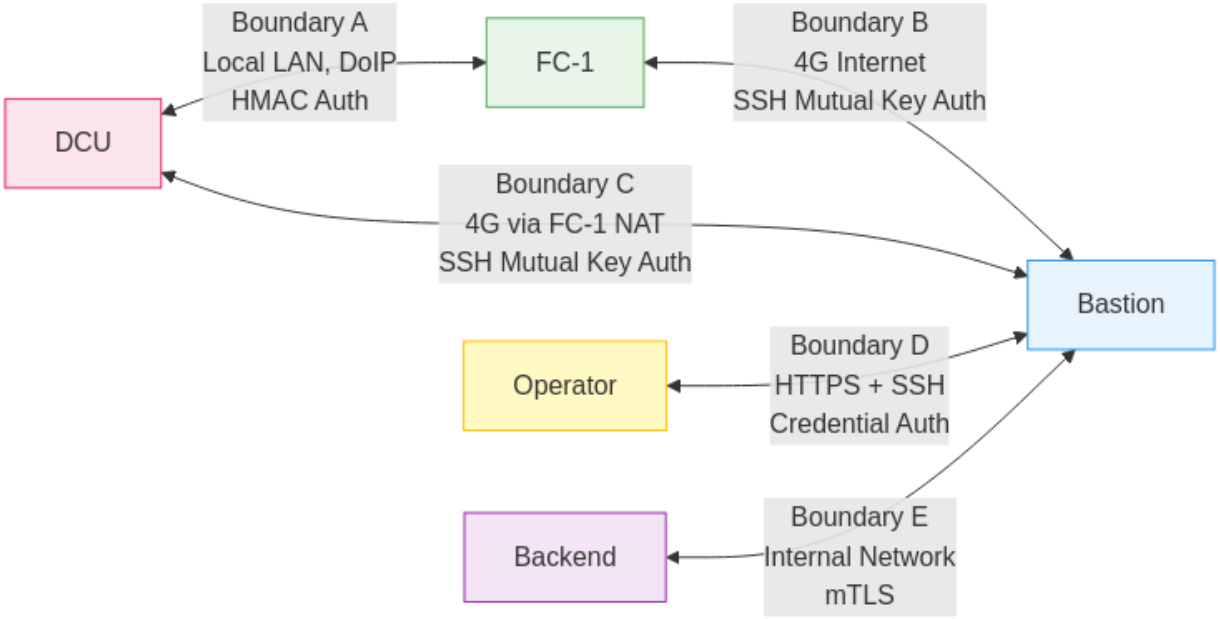

Figure 2: Trust Boundaries

Critical invariant: FC-1 is within the trust boundary for signaling (Boundary A and B) but is **never** in the cryptographic trust chain for DCU sessions (Boundary C). A compromised FC-1 can send a spurious phone-home trigger, but cannot read or intercept the DCU's SSH session.

3.3 Component Ownership

Component	Owner	Language / Environment
DCU DoIP Server	Existing codebase (controlled)	C/C++ or Python
DCU Phone-Home Service Handler	**New — this spec**	C/C++ plugin or Python module
DCU phone-home shell script	**New — this spec**	POSIX sh (BusyBox-compatible)
DCU key provisioning script	**New — this spec**	POSIX sh
FC-1 Signal Relay Agent	**New — this spec**	POSIX sh / Python
Bastion Registration API	**New — this spec**	Python (FastAPI) or Node.js
Bastion Signal Dispatcher	**New — this spec**	Python / shell
Backend CoAP wake endpoint	**New — this spec**	Existing backend, add endpoint

---

4. Security Model and Key Management

4.1 Cryptographic Assets

Asset	Owner	Algorithm	Storage Location	Never Leaves
DCU private key	DCU	ED25519	`/etc/phonehome/id_ed25519` (600, root)	DCU filesystem
DCU public key	DCU → Bastion	ED25519	Registered at Bastion	N/A (public)
FC-1 private key	FC-1	ED25519	`/etc/phonehome/id_ed25519`	FC-1 filesystem
FC-1 public key	FC-1 → Bastion	ED25519	Registered at Bastion	N/A (public)
Bastion host public key	Bastion	ED25519	Pinned in firmware / config	N/A (public)
HMAC shared secret	FC-1 + DCU	HMAC-SHA256	`/etc/phonehome/hmac_secret` (600, root)	Neither device
Provisioning token	Backend → Device	JWT (RS256)	Transient, memory only	N/A (ephemeral)

4.2 Key Generation Policy

- Algorithm: **ED25519** on all devices. Do not use RSA. ED25519 keys are 68 bytes, fast to generate on embedded hardware, and provide 128-bit equivalent security.
- Keys are generated **on the device that will use them**, on first boot, before any network communication.
- Key generation must be seeded from a hardware entropy source. On Linux, read from `/dev/random` (blocking) for key generation, not `/dev/urandom`. Verify `/proc/sys/kernel/random/entropy_` > 256 before generating.
- After generation, the private key file permissions must be set to `0600`, owned by root. Verify this in the provisioning script.

4.3 Bastion Host Key Pinning

The Bastion's ED25519 host public key must be **embedded in the device firmware at build time** and written to `/etc/phonehome/known_hosts` during OS image build. It must not be fetched dynamically. The SSH client must be invoked with `-o StrictHostKeyChecking=yes` and `-o UserKnownHostsFile=/etc/phonehome/known_hosts`.

If the Bastion host key is rotated (planned key rotation), a firmware update must distribute the new host key before the old one expires. Overlap period of at minimum 30 days is required.

4.4 HMAC Shared Secret (FC-1 DCU)

The HMAC secret is a 32-byte random value provisioned at manufacture, identical on both FC-1 and the paired DCU. It authenticates DoIP phone-home trigger messages from FC-1 to DCU, ensuring a spoofed device on the local LAN cannot trigger a phone-home. The secret must be different for every FC-1/DCU pair.

Provisioning method: embedded in the ECU configuration binary at EOL (End of Line) programming, or written to a protected NVM partition.

---

5. Component Specifications

5.1 DCU: Key Generation and Storage

File: `/usr/sbin/phonehome-keygen.sh`
Invocation: Called by systemd service `phonehome-keygen.service` on first boot (runs once, guarded by sentinel file).

```
#!/bin/sh
# phonehome-keygen.sh
# Generates DCU SSH key pair for phone-home function.
# MUST run before any network interfaces are brought up (Before=network.target).
# Complies with ISO 21434 key generation requirements.

set -e

PHONEHOME_DIR="/etc/phonehome"
KEY_FILE="${PHONEHOME_DIR}/id_ed25519"
SENTINEL="${PHONEHOME_DIR}/.keygen_complete"
SERIAL_FILE="/etc/dcu-serial"   # Written at manufacture / EOL programming
LOG_TAG="phonehome-keygen"

log() { logger -t "$LOG_TAG" "$1"; }

# Abort if already done
[ -f "$SENTINEL" ] && { log "Keys already generated. Exiting."; exit 0; }

# Ensure directory exists and is protected
install -d -m 700 -o root -g root "$PHONEHOME_DIR"

# Verify sufficient entropy before generating keys
ENTROPY=$(cat /proc/sys/kernel/random/entropy_avail)
if [ "$ENTROPY" -lt 256 ]; then
    log "ERROR: Insufficient entropy ($ENTROPY bits). Cannot generate keys safely."
    exit 1
fi

# Read device serial number
if [ ! -f "$SERIAL_FILE" ]; then
    log "ERROR: Serial number file not found at $SERIAL_FILE"
    exit 1
fi
DCU_SERIAL=$(cat "$SERIAL_FILE" | tr -d '[:space:]')

# Generate ED25519 key pair (no passphrase; service account usage)
ssh-keygen -t ed25519 -f "$KEY_FILE" -N "" -C "dcu-${DCU_SERIAL}"

# Enforce permissions
chmod 600 "$KEY_FILE"
```

```
chmod 644 "${KEY_FILE}.pub"
chown root:root "$KEY_FILE" "${KEY_FILE}.pub"

# Write sentinel
touch "$SENTINEL"
chmod 600 "$SENTINEL"

log "Key generation complete for DCU serial: $DCU_SERIAL"
log "Public key: $(cat ${KEY_FILE}.pub)"
```

Systemd unit: `/etc/systemd/system/phonehome-keygen.service`

```
[Unit]
Description=DCU Phone-Home SSH Key Generation (one-time)
Before=network-pre.target phonehome-register.service
ConditionPathExists=!/etc/phonehome/.keygen_complete
DefaultDependencies=no

[Service]
Type=oneshot
ExecStart=/usr/sbin/phonehome-keygen.sh
RemainAfterExit=yes
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```

5.2 DCU: Bastion Registration

File: `/usr/sbin/phonehome-register.sh`
Invocation: Called by `phonehome-register.service` on first boot, after key generation and after network is available. Retries until successful. Guarded by sentinel.

```
#!/bin/sh
# phonehome-register.sh
# Registers DCU public key with Bastion via HTTPS.
# FC-1 acts as NAT proxy; this script makes a direct HTTPS call outbound.

set -e

PHONEHOME_DIR="/etc/phonehome"
KEY_FILE="${PHONEHOME_DIR}/id_ed25519"
SENTINEL="${PHONEHOME_DIR}/.registration_complete"
SERIAL_FILE="/etc/dcu-serial"
FC1_SERIAL_FILE="/etc/fc1-serial"          # Written by FC-1 via DoIP at manufacture
BASTION_REG_URL="https://bastion-dev.imatrixsys.com/api/v1/devices/register"
PROVISIONING_TOKEN_FILE="${PHONEHOME_DIR}/provisioning_token"  # Written at manufacture
```

```
CA_CERT="/etc/phonehome/bastion-ca.crt"   # Pinned CA cert, baked into firmware
MAX_RETRIES=10
RETRY_INTERVAL=60
LOG_TAG="phonehome-register"

log() { logger -t "$LOG_TAG" "$1"; }

[ -f "$SENTINEL" ] && { log "Already registered. Exiting."; exit 0; }

DCU_SERIAL=$(cat "$SERIAL_FILE" | tr -d '[:space:]')
FC1_SERIAL=$(cat "$FC1_SERIAL_FILE" | tr -d '[:space:]')
PUBLIC_KEY=$(cat "${KEY_FILE}.pub")
PROV_TOKEN=$(cat "$PROVISIONING_TOKEN_FILE" | tr -d '[:space:]')

attempt=0
while [ $attempt -lt $MAX_RETRIES ]; do
    attempt=$((attempt + 1))
    log "Registration attempt $attempt of $MAX_RETRIES"

    HTTP_STATUS=$(curl -s -o /tmp/reg_response.json -w "%{http_code}" \
        --cacert "$CA_CERT" \
        --max-time 30 \
        -X POST "$BASTION_REG_URL" \
        -H "Authorization: Bearer $PROV_TOKEN" \
        -H "Content-Type: application/json" \
        -d "{
            \"serial\": \"$DCU_SERIAL\",
            \"fc1_serial\": \"$FC1_SERIAL\",
            \"device_type\": \"DCU\",
            \"public_key\": \"$PUBLIC_KEY\"
        }")

    if [ "$HTTP_STATUS" = "200" ] || [ "$HTTP_STATUS" = "201" ]; then
        log "Registration successful (HTTP $HTTP_STATUS)"
        touch "$SENTINEL"
        # Securely delete provisioning token after use
        shred -u "$PROVISIONING_TOKEN_FILE" 2>/dev/null || rm -f "$PROVISIONING_TOKEN_FILE"
        exit 0
    else
        log "Registration failed (HTTP $HTTP_STATUS). Response: $(cat /tmp/reg_response.json)"
        sleep $RETRY_INTERVAL
    fi
done

log "ERROR: Registration failed after $MAX_RETRIES attempts."
exit 1
```

5.3 DCU: Phone-Home SSH Client Script

File: `/usr/sbin/phonehome-connect.sh`
Invocation: Called by the DoIP Phone-Home Service Handler when a valid trigger is received.

```
#!/bin/sh
# phonehome-connect.sh
# Opens a reverse SSH tunnel to the Bastion server.
# Called with: phonehome-connect.sh <bastion_host> <remote_port> <nonce>
#
# Arguments:
#   $1 - Bastion hostname or IP (should match pinned known_hosts entry)
#   $2 - Remote port on Bastion for reverse tunnel (0 = server auto-assigns)
#   $3 - Nonce from the DoIP trigger (logged for audit trail)

set -e

BASTION_HOST="${1:-bastion-dev.imatrixsys.com}"
REMOTE_PORT="${2:-0}"
NONCE="$3"
LOCAL_SSH_PORT=22
PHONEHOME_DIR="/etc/phonehome"
KEY_FILE="${PHONEHOME_DIR}/id_ed25519"
KNOWN_HOSTS="${PHONEHOME_DIR}/known_hosts"
SERIAL_FILE="/etc/dcu-serial"
DCU_SERIAL=$(cat "$SERIAL_FILE" | tr -d '[:space:]')
LOG_TAG="phonehome-connect"
TUNNEL_TIMEOUT=3600   # Max tunnel lifetime in seconds (1 hour)
LOCK_FILE="/var/run/phonehome.lock"

log() { logger -t "$LOG_TAG" "$1"; }

# Prevent concurrent tunnels
if [ -f "$LOCK_FILE" ]; then
    PID=$(cat "$LOCK_FILE")
    if kill -0 "$PID" 2>/dev/null; then
        log "Tunnel already active (PID $PID). Ignoring duplicate trigger."
        exit 0
    fi
fi

echo $$ > "$LOCK_FILE"
trap 'rm -f "$LOCK_FILE"; log "Tunnel closed."' EXIT

log "Phone-home triggered. Serial=$DCU_SERIAL Nonce=$NONCE Bastion=$BASTION_HOST RemotePort=$RE

# Validate key files exist
[ -f "$KEY_FILE" ] || { log "ERROR: Private key not found at $KEY_FILE"; exit 1; }
```

```
[ -f "$KNOWN_HOSTS" ] || { log "ERROR: known_hosts not found at $KNOWN_HOSTS"; exit 1; }

# Open reverse SSH tunnel
# -N: no remote command
# -T: no TTY allocation
# -R: reverse tunnel (remote_port:localhost:local_ssh_port)
# -o ExitOnForwardFailure=yes: fail immediately if port binding fails
# timeout: hard kill after TUNNEL_TIMEOUT seconds

timeout "$TUNNEL_TIMEOUT" ssh \
    -N -T \
    -i "$KEY_FILE" \
    -o StrictHostKeyChecking=yes \
    -o UserKnownHostsFile="$KNOWN_HOSTS" \
    -o ExitOnForwardFailure=yes \
    -o ServerAliveInterval=30 \
    -o ServerAliveCountMax=3 \
    -o BatchMode=yes \
    -o ConnectTimeout=30 \
    -R "${REMOTE_PORT}:localhost:${LOCAL_SSH_PORT}" \
    "phonehome-${DCU_SERIAL}@${BASTION_HOST}" \
    || log "SSH tunnel exited with code $?"
```

Notes for developers: - The SSH username format `phonehome-<DCU_SERIAL>` is how the Bastion identifies the device. The Bastion's `authorized_keys` for each device user is configured with `command=""` and `restrict` options to limit what the connecting device can do. - `REMOTE_PORT=0` instructs the SSH server to auto-assign an ephemeral port. The Bastion's signal dispatcher must read the assigned port from the SSH server and communicate it back to the operator. See Section 5.7. - `timeout` must be available on the target. On BusyBox, use `timeout` applet. On Ubuntu, it is in `coreutils`.

5.4 DCU: DoIP Phone-Home Service Handler

This is the **new code** to be integrated into the existing DCU DoIP server. See Section 6 for full DoIP integration details.

UDS RoutineControl Handler Flow The following flowchart shows the complete dispatch and validation logic for all three routine identifiers (0xF0A0 trigger, 0xF0A1 provisioning, 0xF0A2 status query):

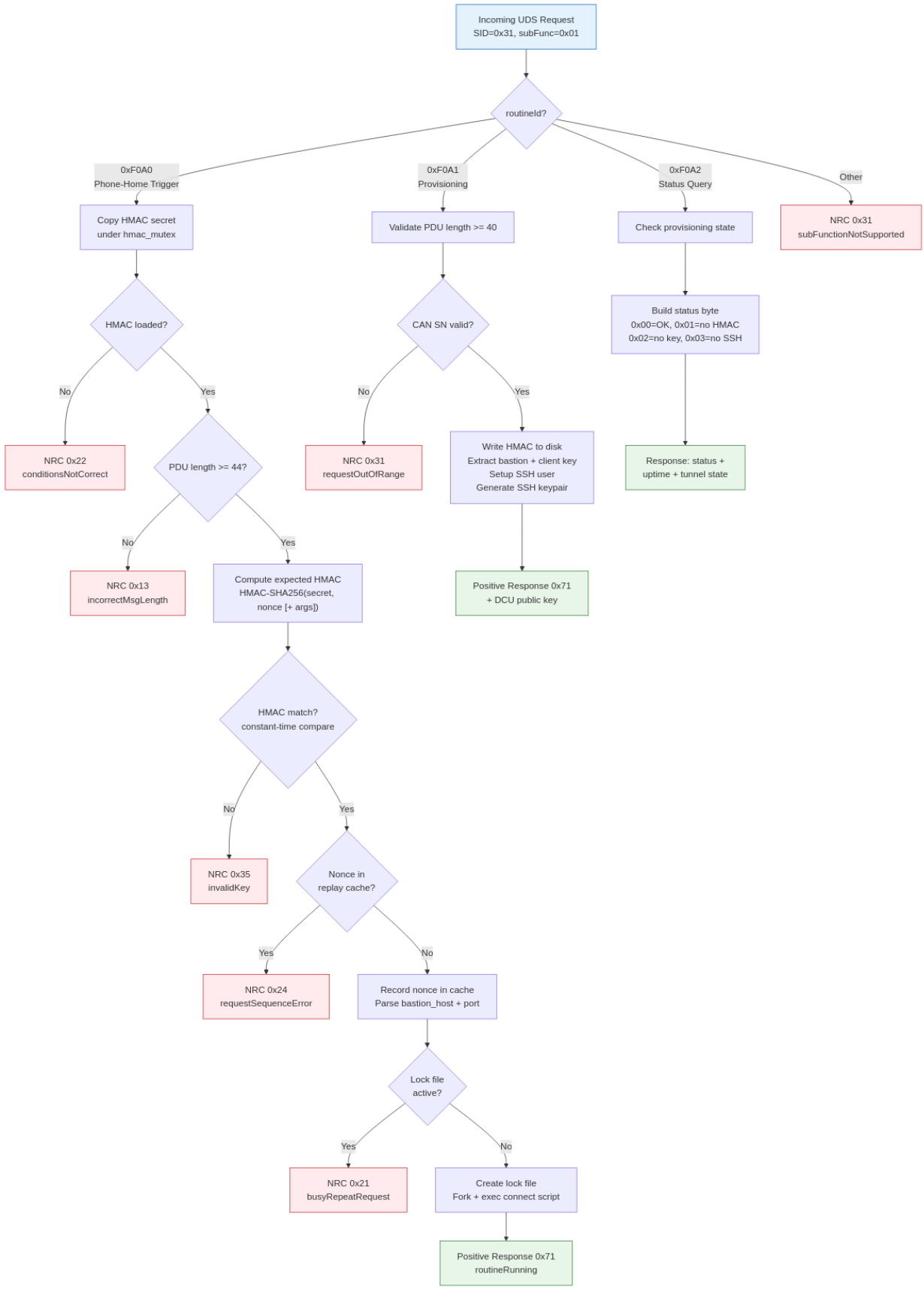

Figure 3: UDS Handler Flowchart

Logical specification (language-agnostic, adapt to your DoIP server's plugin/handler architecture):

```
Handler: PhoneHomeRoutineControl

Triggers on:
    SID = 0x31 (RoutineControl)
    subFunction = 0x01 (startRoutine)
    routineIdentifier = 0xF0A0

Request PDU format (after UDS header):
    Byte 0:     SID (0x31)
    Byte 1:     subFunction (0x01)
    Bytes 2-3:  routineIdentifier (0xF0, 0xA0)
    Bytes 4-11: Nonce (8 bytes, random, used for replay protection)
    Bytes 12-43: HMAC-SHA256 digest (32 bytes)
                 HMAC key: shared secret from /etc/phonehome/hmac_secret
                 HMAC input: Nonce bytes (bytes 4-11)

Handler logic:
    1. Extract nonce (bytes 4-11)
    2. Extract HMAC digest (bytes 12-43)
    3. Compute expected HMAC = HMAC-SHA256(hmac_secret, nonce)
    4. Compare expected HMAC to received HMAC using constant-time comparison
       (to prevent timing attacks)
    5. If HMAC mismatch: respond with NRC 0x35 (invalidKey) and log security event
    6. Check nonce against replay cache (see below)
    7. If nonce replay detected: respond with NRC 0x24 (requestSequenceError)
    8. Add nonce to replay cache with TTL = 300 seconds
    9. Extract optional bastion_host (bytes 44+, UTF-8, null-terminated, max 253 chars)
       If absent: use compiled-in default from config
    10. Extract optional remote_port (2 bytes at offset 44+len(bastion_host)+1)
        If absent: use 0 (auto-assign)
    11. Spawn /usr/sbin/phonehome-connect.sh <bastion_host> <remote_port> <nonce_hex>
        as a detached child process (do not wait for SSH to complete)
    12. Respond with positive response: SID+0x40=0x71, subFunction 0x01,
        routineIdentifier 0xF0A0, routineStatus=0x02 (routineRunning)

Replay Cache:
    In-memory circular buffer, capacity 64 entries.
    Each entry: { nonce: bytes[8], expiry: unix_timestamp }
    On lookup: evict expired entries, then search for matching nonce.
    Thread-safe: protect with mutex if DoIP server is multi-threaded.

Positive Response PDU:
    Byte 0: 0x71
    Byte 1: 0x01
    Bytes 2-3: 0xF0 0xA0
```

```
    Byte 4: 0x02  (routineRunning - tunnel is initiating asynchronously)

NRC Response PDU (on error):
    Byte 0: 0x7F
    Byte 1: 0x31
    Byte 2: <NRC code>
```

5.5 FC-1: Signal Relay and DCU Provisioning

File: `Fleet-Connect-1/remote_access/remote_access.c` **Invocation:** Integrated into the FC-1 main loop. Runs automatically — no external script or manual trigger required.

5.5.1 HMAC Secret Management The HMAC secret is auto-generated on first boot if not present:

```
// In init_load_hmac_secret():
// 1. Try to load from /usr/qk/etc/sv/FC-1/hmac_secret
// 2. If not found: generate 32 bytes from /dev/urandom
// 3. Write atomically (temp file + rename) with mode 0600
// 4. Load into ctx.hmac_secret[32]
```

Storage: `/usr/qk/etc/sv/FC-1/hmac_secret` (yaffs2, persistent across reboots) **Permissions:** 0600 (root read/write only)

5.5.2 DCU Provisioning (routineId 0xF0A1) After the DoIP TCP connection is established and the CAN controller serial number is available, the FC-1 sends a 40-byte UDS provisioning PDU:

Offset	Length	Field
0	1	SID: 0x31 (RoutineControl)
1	1	Sub-function: 0x01 (startRoutine)
2-3	2	Routine ID: 0xF0A1 (provision)
4-7	4	CAN controller serial number (big-endian uint32)
8-39	32	HMAC-SHA256 shared secret

The DCU stores the secret atomically and responds with a 5-byte positive response: `0x71 0x01 0xF0 0xA1 0x00` (routineAccepted).

Provisioning runs once per FC-1 power cycle (max 3 UDS-level retries; transport errors do not count against the retry limit).

5.5.3 DCU Phone-Home Relay (routineId 0xF0A0) When the FC-1 receives a CoAP POST to `/remote_call_home/{can_sn}` from the bastion (via the iMatrix CoAP infrastructure), it relays the trigger to the DCU:

Offset	Length	Field
0	1	SID: 0x31 (RoutineControl)
1	1	Sub-function: 0x01 (startRoutine)
2-3	2	Routine ID: 0xF0A0 (phone-home trigger)
4-11	8	Nonce (random, from /dev/urandom)
12-43	32	HMAC-SHA256(secret, nonce)

The relay is rate-limited to once per 60 seconds. Uses a static `doip_client_t` to avoid stack allocation on embedded. Fire-and-forget: logs result, no retry.

5.5.4 Persistent Tunnel After bastion registration, the FC-1 automatically establishes a persistent reverse SSH tunnel using `dbclient` (Dropbear SSH client):

- Reverse port: `127.0.0.1:<tunnel_port>` → `127.0.0.1:22222` (FC-1 SSH)
- Keepalive: 25 seconds
- TTL: 3600 seconds (auto-rotated)
- Host key: always TOFU (-y), stale known_hosts entries auto-cleared
- On disconnect: auto-reconnect after 60-second cooldown
- On TTL expiry: immediate reconnect (tunnel rotation)

Tunnel script: `Fleet-Connect-1/remote_access/tunnel.sh` (embedded in binary, extracted to `/usr/qk/etc/sv/FC-1/scripts/tunnel.sh` at runtime)

5.6 Bastion: Registration API

Technology: Python 3 + FastAPI (or equivalent). Runs as a service on the Bastion.

```python
# registration_api.py
# Bastion-side device registration endpoint.

from fastapi import FastAPI, HTTPException, Depends, Header
from pydantic import BaseModel, constr
import sqlite3
import re
import subprocess
import os

app = FastAPI()
DB_PATH = "/var/lib/phonehome/devices.db"
AUTHORIZED_KEYS_DIR = "/etc/phonehome/authorized_keys"  # One file per device user

class RegistrationRequest(BaseModel):
    serial: constr(pattern=r'^[A-Z0-9\-]{6,32}$')
    fc1_serial: constr(pattern=r'^[A-Z0-9\-]{6,32}$')
    device_type: constr(pattern=r'^(DCU|FC1)$')
    public_key: str  # Will be validated as ED25519 public key
```

```python
def validate_ed25519_pubkey(key: str) -> bool:
    """Validates that the string is a valid ED25519 SSH public key."""
    parts = key.strip().split()
    if len(parts) < 2 or parts[0] != "ssh-ed25519":
        return False
    import base64
    try:
        data = base64.b64decode(parts[1])
        return len(data) == 51  # ED25519 public key is always 51 bytes encoded
    except Exception:
        return False

def verify_provisioning_token(authorization: str = Header(...)):
    """
    Validates JWT provisioning token.
    Token is scoped to a specific serial number and is single-use.
    Implementation detail: use python-jose or PyJWT.
    """
    # TODO: implement JWT validation with RS256, check 'sub' claim matches serial,
    # check 'exp' claim, check 'jti' against used-token store (prevent reuse).
    if not authorization.startswith("Bearer "):
        raise HTTPException(status_code=401, detail="Invalid authorization header")
    token = authorization[7:]
    # validate_jwt(token)  # implement this
    return token

@app.post("/api/v1/devices/register", status_code=201)
def register_device(req: RegistrationRequest, token: str = Depends(verify_provisioning_token))
    if not validate_ed25519_pubkey(req.public_key):
        raise HTTPException(status_code=400, detail="Invalid ED25519 public key")

    # Write to database
    con = sqlite3.connect(DB_PATH)
    con.execute("""
        INSERT OR REPLACE INTO devices (serial, fc1_serial, device_type, public_key, registere
        VALUES (?, ?, ?, ?, datetime('now'))
    """, (req.serial, req.fc1_serial, req.device_type, req.public_key.strip()))
    con.commit()
    con.close()

    # Write authorized_keys file for this device's SSH user
    username = f"phonehome-{req.serial}"
    key_path = os.path.join(AUTHORIZED_KEYS_DIR, username)
    restricted_key = (
        f'restrict,command="" {req.public_key.strip()}\n'
    )
    os.makedirs(AUTHORIZED_KEYS_DIR, mode=0o700, exist_ok=True)
```

```
    with open(key_path, 'w') as f:
        f.write(restricted_key)
    os.chmod(key_path, 0o600)

    return {"status": "registered", "serial": req.serial}
```

Database schema:

```
-- /var/lib/phonehome/schema.sql
CREATE TABLE IF NOT EXISTS devices (
    serial          TEXT PRIMARY KEY,
    fc1_serial      TEXT NOT NULL,
    device_type     TEXT NOT NULL CHECK(device_type IN ('DCU', 'FC1')),
    public_key      TEXT NOT NULL,
    registered_at   TEXT NOT NULL,
    last_seen       TEXT,
    status          TEXT DEFAULT 'registered'
);

CREATE TABLE IF NOT EXISTS phone_home_log (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    serial          TEXT NOT NULL,
    operator_id     TEXT NOT NULL,
    requested_at    TEXT NOT NULL,
    connected_at    TEXT,
    disconnected_at TEXT,
    remote_port     INTEGER,
    outcome         TEXT
);

CREATE TABLE IF NOT EXISTS used_provisioning_tokens (
    jti             TEXT PRIMARY KEY,
    serial          TEXT NOT NULL,
    used_at         TEXT NOT NULL
);
```

5.7 Bastion: Signal Dispatcher

The Bastion signal dispatcher receives a wake request from the backend, uses the FC-1's existing reverse tunnel to call `phonehome-relay.sh` on FC-1, and monitors for the DCU's inbound reverse tunnel.

```
# signal_dispatcher.py

import subprocess
import time
import sqlite3
import logging
```

```python
DB_PATH = "/var/lib/phonehome/devices.db"
SSH_KEY = "/etc/phonehome/bastion_relay_key"   # Bastion's key for reaching FC-1 via tunnel
LOG = logging.getLogger("signal_dispatcher")

def get_fc1_tunnel_port(fc1_serial: str) -> int:
    """
    Returns the local port on the Bastion that FC-1's reverse tunnel is listening on.
    This is maintained by a separate tunnel-monitor process that reads sshd logs
    or uses the SSH multiplexer API.
    """
    # Implementation: query a local state file / Redis / SQLite that is
    # updated by the tunnel-monitor daemon.
    raise NotImplementedError("Implement tunnel port lookup for FC-1")

def dispatch_wake(dcu_serial: str, operator_id: str) -> dict:
    """
    Sends a phone-home signal to the specified DCU via FC-1's reverse tunnel.
    Returns the assigned remote port for the operator to connect to.
    """
    con = sqlite3.connect(DB_PATH)
    row = con.execute(
        "SELECT fc1_serial FROM devices WHERE serial=? AND device_type='DCU'",
        (dcu_serial,)
    ).fetchone()
    con.close()

    if not row:
        raise ValueError(f"Unknown DCU serial: {dcu_serial}")

    fc1_serial = row[0]
    fc1_port = get_fc1_tunnel_port(fc1_serial)

    LOG.info(f"Dispatching wake for DCU {dcu_serial} via FC-1 {fc1_serial} on port {fc1_port}")

    # SSH into FC-1 via its reverse tunnel and execute phonehome-relay.sh
    result = subprocess.run([
        "ssh",
        "-i", SSH_KEY,
        "-o", "StrictHostKeyChecking=yes",
        "-o", "UserKnownHostsFile=/etc/phonehome/fc1_known_hosts",
        "-o", "BatchMode=yes",
        "-o", "ConnectTimeout=15",
        "-p", str(fc1_port),
        f"relay-{fc1_serial}@localhost",
        f"/usr/sbin/phonehome-relay.sh {dcu_serial} bastion-dev.imatrixsys.com 0"
    ], capture_output=True, text=True, timeout=30)
```

```
    if result.returncode != 0:
        raise RuntimeError(f"Relay failed: {result.stderr}")

    # Wait for DCU's reverse tunnel to appear (poll for up to 60s)
    assigned_port = wait_for_dcu_tunnel(dcu_serial, timeout=60)
    if not assigned_port:
        raise TimeoutError(f"DCU {dcu_serial} did not connect within 60 seconds")

    # Log the event
    con = sqlite3.connect(DB_PATH)
    con.execute("""
        INSERT INTO phone_home_log (serial, operator_id, requested_at, connected_at, remote_por
        VALUES (?, ?, datetime('now'), datetime('now'), ?, 'connected')
    """, (dcu_serial, operator_id, assigned_port))
    con.commit()
    con.close()

    return {"dcu_serial": dcu_serial, "bastion_host": "bastion-dev.imatrixsys.com", "port": ass

def wait_for_dcu_tunnel(dcu_serial: str, timeout: int = 60) -> int:
    """
    Polls the SSH server (via ss/netstat or sshd log parsing) to detect
    the DCU's reverse tunnel port becoming active.
    Returns the port number or None on timeout.
    """
    # Implementation: the SSH server is configured with AuthorizedKeysFile pointing
    # to per-device files. Use `ss -tlnp` or parse `/var/log/auth.log` to detect
    # when phonehome-<dcu_serial> user connects and which port they bind.
    # Alternatively: use OpenSSH's StreamLocalBindUnlink + a Unix socket approach.
    raise NotImplementedError("Implement DCU tunnel port detection")
```

5.8 Backend/CoAP Server: Operator Trigger API

Add this endpoint to your existing backend. It is called when the operator clicks the "Connect" button in the web UI.

```
POST /api/v1/phonehome/wake

Authorization: Bearer <operator JWT>

Request body:
{
    "dcu_serial": "DCU-XXXXXXXX"
}

Response (202 Accepted, async):
{
```

```
    "status": "dispatched",
    "dcu_serial": "DCU-XXXXXXXX",
    "poll_url": "/api/v1/phonehome/status/DCU-XXXXXXXX"
}

Response (200 OK, when tunnel is ready):
{
    "status": "ready",
    "dcu_serial": "DCU-XXXXXXXX",
    "bastion_host": "bastion-dev.imatrixsys.com",
    "port": 34521,
    "expires_at": "2024-01-01T12:00:00Z"
}
```

The backend calls `signal_dispatcher.dispatch_wake()` synchronously or asynchronously, then the UI polls `/status/` until `"status": "ready"` is returned.

---

6. DoIP Integration

6.1 Custom Routine Control Service

The phone-home trigger uses UDS service 0x31 (RoutineControl) with a vendor-specific `routineIdentifier` in the range 0xF000–0xFFFF. The assigned identifier for this function is **0xF0A0**.

Field	Value	Notes
SID	0x31	RoutineControl
subFunction	0x01	startRoutine
routineIdentifier	0xF0A0	Vendor-specific, phone-home
NRC on HMAC fail	0x35	invalidKey
NRC on replay	0x24	requestSequenceError
NRC on busy	0x21	busyRepeatRequest (tunnel already active)

6.2 Integration into Existing DoIP Server

The following describes how to integrate the phone-home handler into a typical C/C++ DoIP server. Adapt to your actual codebase.

Step 1: Register the handler.

Locate the function in your DoIP server that dispatches incoming UDS requests based on SID. This is typically a switch/case or dispatch table. Add a registration call:

```
// In your UDS dispatcher initialization:
uds_register_handler(
    SID_ROUTINE_CONTROL,          // 0x31
    SUB_FUNC_START_ROUTINE,       // 0x01
    ROUTINE_ID_PHONEHOME,         // 0xF0A0
```

```
    phonehome_routine_handler     // function pointer
);
```

Step 2: Implement the handler function.

```
// phonehome_handler.c

#include <stdint.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <openssl/hmac.h>
#include "uds_types.h"
#include "phonehome_handler.h"

#define NONCE_LEN       8
#define HMAC_LEN        32
#define PAYLOAD_OFFSET  4   // After SID + subFunc + routineIdentifier (2 bytes)
#define NONCE_OFFSET    PAYLOAD_OFFSET
#define HMAC_OFFSET     (NONCE_OFFSET + NONCE_LEN)
#define ARGS_OFFSET     (HMAC_OFFSET + HMAC_LEN)
#define REPLAY_CACHE_SIZE 64
#define NONCE_TTL_SECS  300

typedef struct {
    uint8_t nonce[NONCE_LEN];
    time_t  expiry;
} replay_entry_t;

static replay_entry_t replay_cache[REPLAY_CACHE_SIZE];
static int            replay_index = 0;
static pthread_mutex_t replay_mutex = PTHREAD_MUTEX_INITIALIZER;

static uint8_t hmac_secret[32];     // Loaded at startup from /etc/phonehome/hmac_secret
static int     hmac_secret_loaded = 0;

// Call this during DoIP server initialization
int phonehome_init(void) {
    FILE *f = fopen("/etc/phonehome/hmac_secret", "rb");
    if (!f) return -1;
    size_t n = fread(hmac_secret, 1, sizeof(hmac_secret), f);
    fclose(f);
    if (n != sizeof(hmac_secret)) return -1;
    hmac_secret_loaded = 1;
    return 0;
}

static int check_and_record_nonce(const uint8_t *nonce) {
    time_t now = time(NULL);
```

```
    pthread_mutex_lock(&replay_mutex);
    for (int i = 0; i < REPLAY_CACHE_SIZE; i++) {
        if (replay_cache[i].expiry > now &&
            memcmp(replay_cache[i].nonce, nonce, NONCE_LEN) == 0) {
            pthread_mutex_unlock(&replay_mutex);
            return -1;  // Replay detected
        }
    }
    // Evict oldest entry and record new nonce
    replay_cache[replay_index % REPLAY_CACHE_SIZE].expiry = now + NONCE_TTL_SECS;
    memcpy(replay_cache[replay_index % REPLAY_CACHE_SIZE].nonce, nonce, NONCE_LEN);
    replay_index++;
    pthread_mutex_unlock(&replay_mutex);
    return 0;
}

uds_response_t phonehome_routine_handler(const uint8_t *req, size_t req_len) {
    if (!hmac_secret_loaded) return uds_nrc(0x31, 0x22);  // conditionsNotCorrect
    if (req_len < ARGS_OFFSET) return uds_nrc(0x31, 0x13);  // incorrectMessageLength

    const uint8_t *nonce = req + NONCE_OFFSET;
    const uint8_t *rx_hmac = req + HMAC_OFFSET;

    // Compute expected HMAC
    uint8_t exp_hmac[HMAC_LEN];
    unsigned int exp_len = HMAC_LEN;
    HMAC(EVP_sha256(), hmac_secret, sizeof(hmac_secret),
         nonce, NONCE_LEN, exp_hmac, &exp_len);

    // Constant-time comparison (prevents timing side-channel)
    if (CRYPTO_memcmp(rx_hmac, exp_hmac, HMAC_LEN) != 0) {
        syslog(LOG_WARNING, "phonehome: HMAC verification failed - possible spoofing attempt")
        return uds_nrc(0x31, 0x35);  // invalidKey
    }

    // Replay protection
    if (check_and_record_nonce(nonce) != 0) {
        syslog(LOG_WARNING, "phonehome: Replayed nonce detected");
        return uds_nrc(0x31, 0x24);  // requestSequenceError
    }

    // Parse optional args: bastion_host (null-terminated) + port (uint16 BE)
    char bastion_host[254] = "bastion-dev.imatrixsys.com";  // default
    uint16_t remote_port = 0;
    if (req_len > ARGS_OFFSET) {
        size_t remaining = req_len - ARGS_OFFSET;
        const uint8_t *args = req + ARGS_OFFSET;
        size_t host_len = strnlen((const char *)args, remaining < 253 ? remaining : 253);
```

```
        if (host_len > 0 && host_len < 254) {
            memcpy(bastion_host, args, host_len);
            bastion_host[host_len] = '\0';
            size_t port_offset = host_len + 1;
            if (remaining >= port_offset + 2) {
                remote_port = (args[port_offset] << 8) | args[port_offset + 1];
            }
        }
    }

    // Format nonce as hex string for logging
    char nonce_hex[NONCE_LEN * 2 + 1];
    for (int i = 0; i < NONCE_LEN; i++)
        snprintf(nonce_hex + i*2, 3, "%02x", nonce[i]);

    syslog(LOG_INFO, "phonehome: Trigger received. Bastion=%s Port=%u Nonce=%s",
           bastion_host, remote_port, nonce_hex);

    // Spawn phone-home script as detached process
    char port_str[6], nonce_str[NONCE_LEN * 2 + 1];
    snprintf(port_str, sizeof(port_str), "%u", remote_port);
    memcpy(nonce_str, nonce_hex, sizeof(nonce_hex));

    pid_t pid = fork();
    if (pid == 0) {
        // Child: detach and exec
        setsid();
        execl("/usr/sbin/phonehome-connect.sh", "phonehome-connect.sh",
              bastion_host, port_str, nonce_str, NULL);
        _exit(1);
    } else if (pid < 0) {
        syslog(LOG_ERR, "phonehome: fork() failed");
        return uds_nrc(0x31, 0x22);  // conditionsNotCorrect
    }

    // Positive response: routineRunning
    uint8_t resp[] = { 0x71, 0x01, 0xF0, 0xA0, 0x02 };
    return uds_positive_response(resp, sizeof(resp));
}
```

Step 3: Add to your build system.

```
# Add to your Makefile or CMakeLists.txt
SRCS += phonehome_handler.c
LDFLAGS += -lssl -lcrypto  # OpenSSL for HMAC
```

Step 4: Call `phonehome_init()` during startup.

```
// In your main server initialization sequence:
if (phonehome_init() != 0) {
```

```
    syslog(LOG_ERR, "phonehome: Failed to load HMAC secret. Phone-home disabled.");
    // Do not abort the entire server - degrade gracefully.
}
```

6.3 Replay Protection and HMAC

The HMAC and nonce mechanism protects against two threat vectors:

1. **Spoofed trigger:** A rogue device on the vehicle LAN sends a crafted DoIP RoutineControl to trigger an unauthorized phone-home. Prevented by HMAC — attacker does not know the shared secret.
2. **Replay attack:** An attacker captures a legitimate DoIP trigger frame and retransmits it later. Prevented by nonce replay cache — the same 8-byte nonce cannot be used twice within 300 seconds.

The 8-byte nonce provides sufficient entropy for this use case. The 300-second TTL is intentionally conservative; the transit time for a DoIP frame on a local LAN is sub-millisecond.

---

7. Provisioning Flow

Provisioning is **fully automatic** — no manufacturing station or manual steps are required. All cryptographic material is generated at first boot and exchanged over authenticated channels.

7.1 Zero-Touch Provisioning Sequence

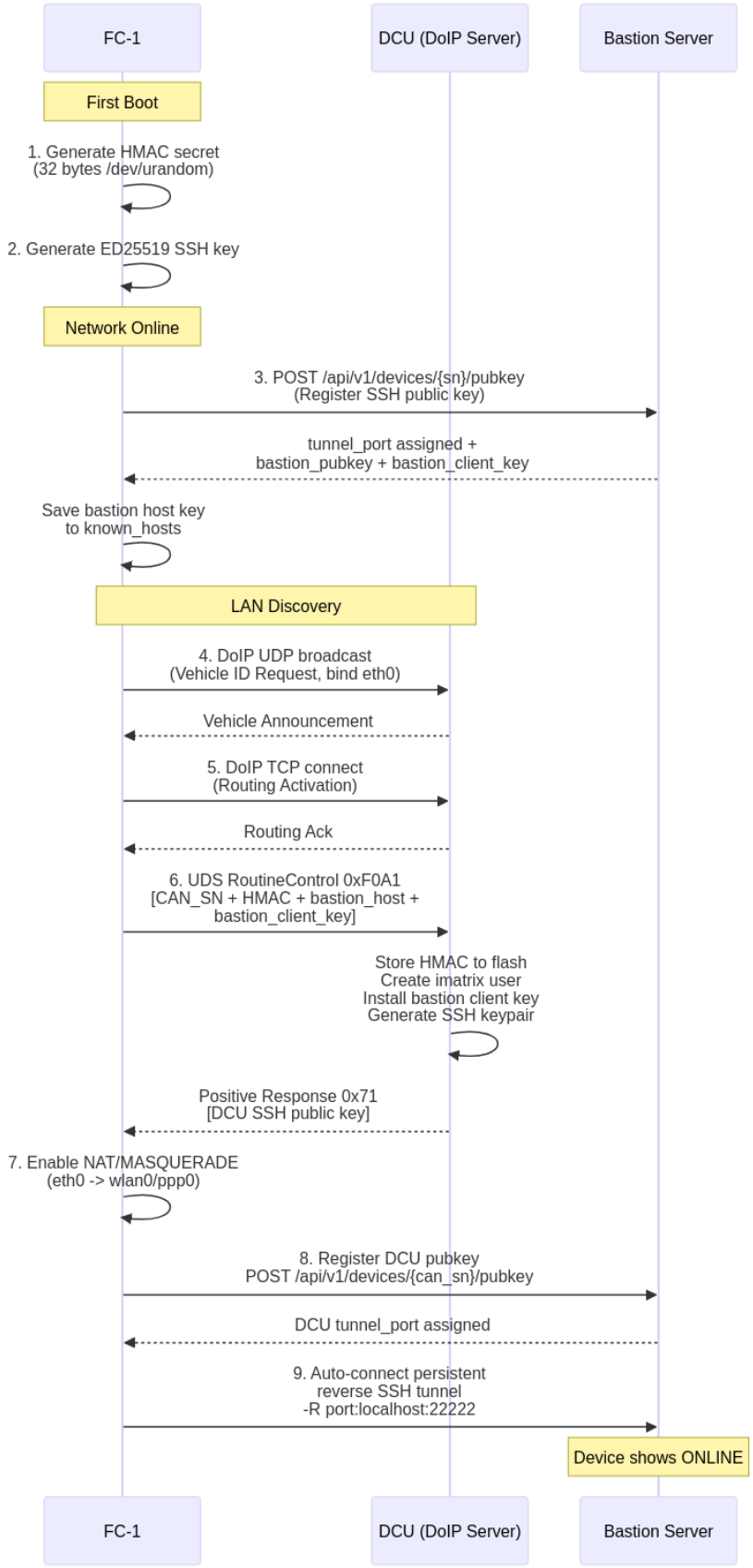

Figure 4: Provisioning Flow

7.2 Provisioning Details

Step	Component	Trigger	Storage	Persistence
HMAC generation	FC-1 `remote_access.c`	First boot (file not found)	`/usr/qk/etc/sv/FC-1/hmac_secret`	Cache/flash
SSH key generation	FC-1 `remote_access.c`	First boot (file not found)	`/usr/qk/etc/sv/FC-1/tunnel-key`	Cache/flash
Bastion registration	FC-1 `remote_access.c`	INITIALIZING state	Bastion database	Permanent
DoIP discovery	FC-1 `doip_process.c`	eth0 interface has IP	In-memory `g_doip` context	Per-boot
HMAC delivery to DCU	FC-1 `remote_access.c`	DoIP connected + HMAC loaded + CAN SN available	DCU `/etc/phonehome/hmac_secret`	DCU flash
NAT/MASQUERADE	FC-1 `connection_sharing.c`	After successful DCU provisioning	iptables rules + ip_forward	Per-boot (rules persist via iptables-save)
Persistent tunnel	FC-1 `remote_access.c`	After bastion registration	Active SSH connection	Maintained with auto-reconnect

7.3 Provisioning State Machine

The following state diagram shows how the DCU transitions between provisioning states and how the FC-1 health check (0xF0A2) drives automatic re-provisioning:

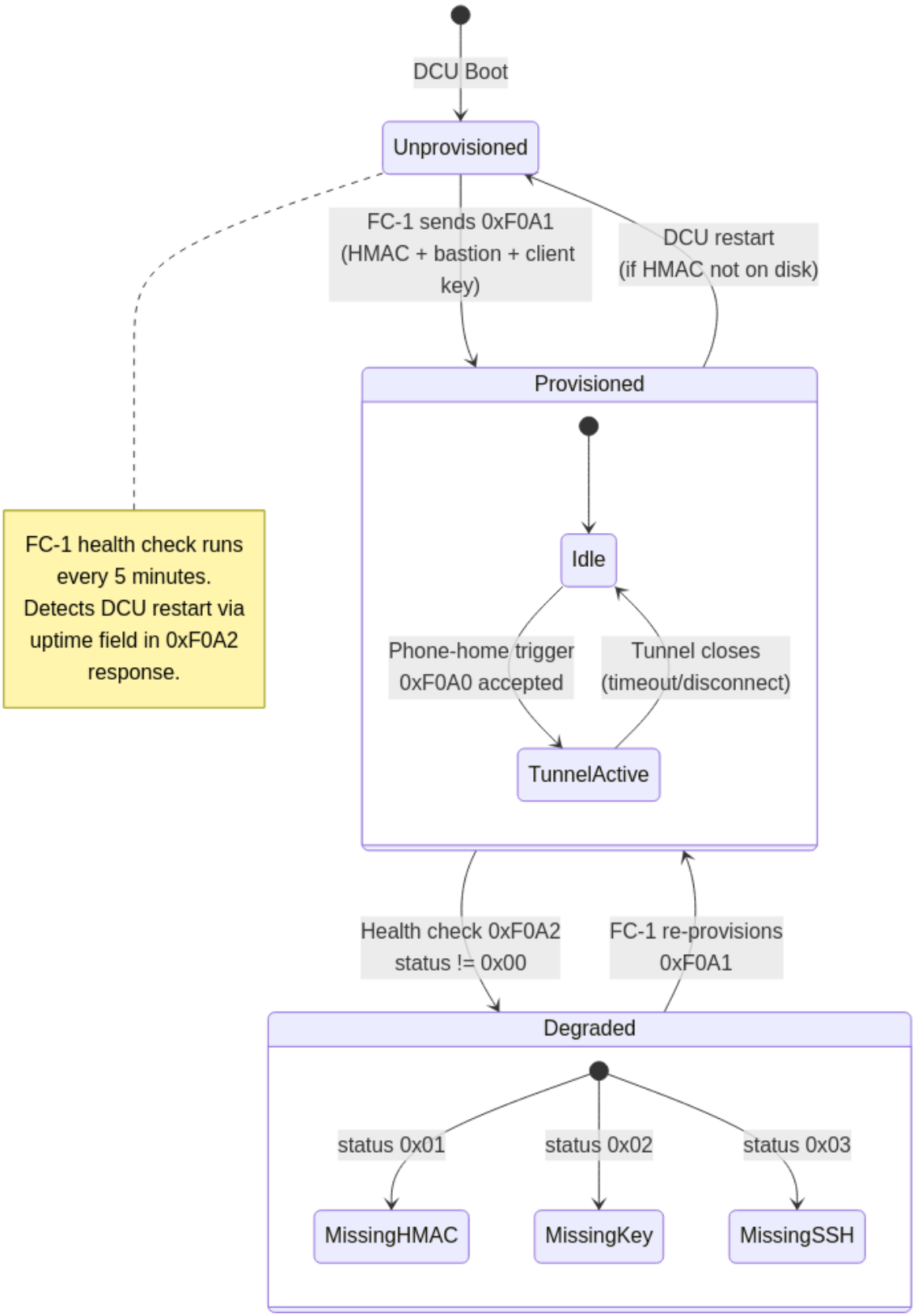

Figure 5: Health Check State Machine

7.4 Re-Provisioning Behavior

- **FC-1 reboot:** HMAC reloaded from flash. DCU re-provisioned on next DoIP connection. Tunnel re-established automatically.
- **DCU reboot (only):** HMAC reloaded from DCU flash. No re-provisioning needed unless HMAC file was lost.
- **HMAC file deleted on FC-1:** Auto-generated on next boot. DCU re-provisioned with the new secret.
- **HMAC file deleted on DCU:** FC-1 re-provisions on next DoIP connection (once per power cycle).
- **Bastion host key change:** Stale key auto-cleared from known_hosts before each tunnel connection. TOFU (-y) accepts the new key.

7.4 Prerequisites

The only requirements for provisioning to succeed:

1. FC-1 has network connectivity (WiFi or cellular) to reach the bastion server
2. DCU is running the DoIP server on the same LAN as FC-1 (eth0, 192.168.7.0/24)
3. CAN controller has provided a serial number (non-zero)

No manufacturing station, no manual configuration, no provisioning tokens.

---

8. Operational Phone-Home Flow

Step-by-step with timing expectations:

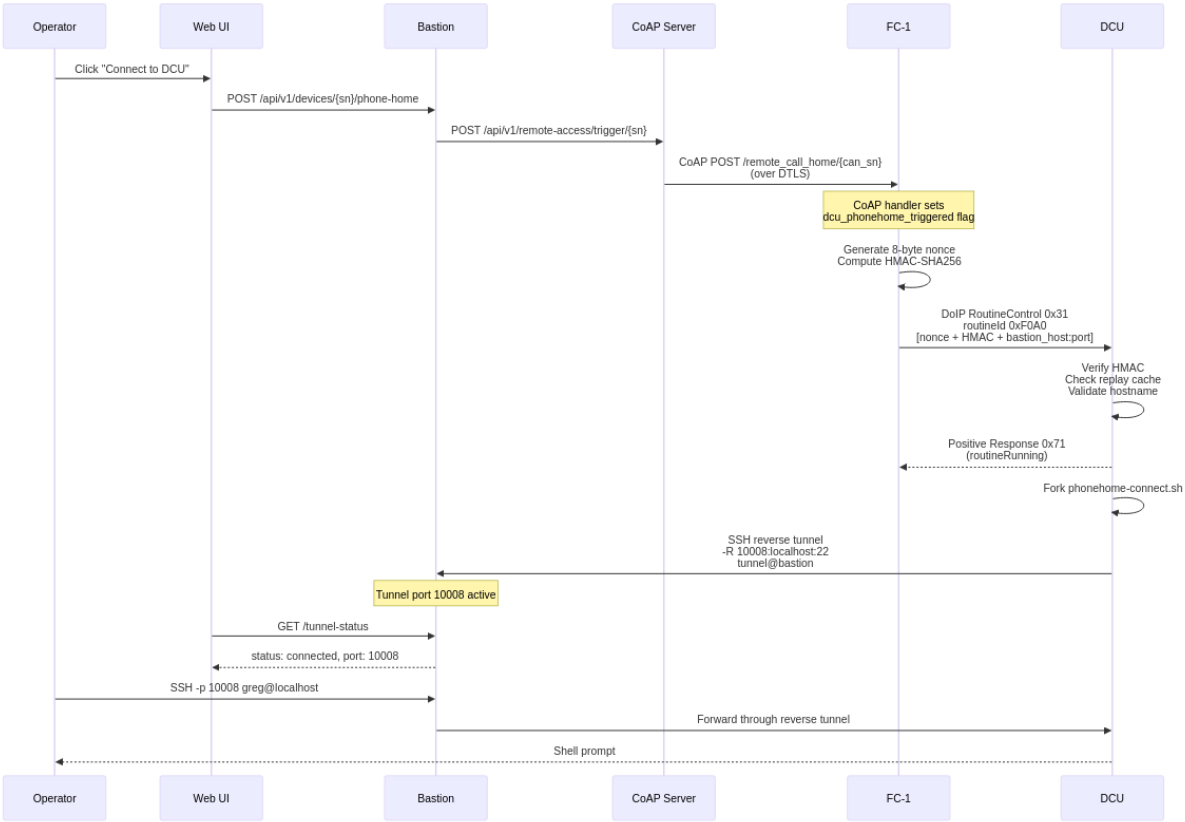

Figure 6: Phone-Home Operational Flow

Expected total latency: 3–8 seconds from button click to operator prompt.

---

9. Configuration Reference

9.1 DCU Configuration File: `/etc/phonehome/phonehome.conf`

```
# /etc/phonehome/phonehome.conf
BASTION_HOST=bastion-dev.imatrixsys.com
BASTION_PORT=22
TUNNEL_TIMEOUT=3600
RETRY_ON_DISCONNECT=0
LOG_LEVEL=info
DOIP_SOURCE_ADDR=0x0001
```

9.2 FC-1 Configuration

The FC-1 does NOT use a configuration file for phone-home. All parameters are derived automatically from the system:

Parameter	Source	Value
Bastion host	iMatrix CoAP server (`imx_get_bastion_url()`)	`bastion-dev.imatrixsys.com`
Bastion port	iMatrix CoAP server (`imx_get_bastion_port()`)	`22`
Tunnel port	Assigned by bastion during registration	e.g., `10007`
HMAC secret	Auto-generated on first boot	`/usr/qk/etc/sv/FC-1/hmac_secret`
SSH key	Auto-generated on first boot	`/usr/qk/etc/sv/FC-1/tunnel-key`
DCU IP	Discovered via DoIP UDP broadcast on eth0	e.g., `192.168.7.101`
CAN SN	Provided by CAN controller subsystem	e.g., `287906454`
Device serial	From device_config (set via `qr` command)	e.g., `0799874683`

9.3 Bastion SSH Server Configuration (`/etc/ssh/sshd_config` additions)

```
# Separate match block for phone-home device users
Match User phonehome-*
    AllowTcpForwarding remote
    X11Forwarding no
    PermitTTY no
    PermitTunnel no
    GatewayPorts no
    AuthorizedKeysFile /etc/phonehome/authorized_keys/%u
    ForceCommand /bin/false

# Separate match block for FC-1 relay users
Match User relay-*
    AllowTcpForwarding remote
    X11Forwarding no
    PermitTTY no
    AuthorizedKeysFile /etc/phonehome/authorized_keys/%u
    ForceCommand /usr/sbin/phonehome-relay-gate.sh
```

---

10. Error Handling and Edge Cases

Scenario	Detection	Response
DCU has no network (4G down)	SSH connect timeout (30s)	Log failure; DoIP handler already responded routineRunning; backend polls and times out; operator sees "Connection failed"

Scenario	Detection	Response
HMAC mismatch on DoIP trigger	HMAC comparison fails	NRC 0x35; security event logged; no tunnel initiated
Nonce replay on DoIP trigger	Nonce found in replay cache	NRC 0x24; event logged
Tunnel already active	Lock file present with live PID	phonehome-connect.sh exits immediately; DoIP handler checks lock before fork and returns NRC 0x21
Bastion host key mismatch	StrictHostKeyChecking=yes fails	SSH exits non-zero; logged; no connection; security event
FC-1 offline	Dispatcher SSH command fails	dispatch_wake throws; backend returns 503 to UI
DCU first boot, not yet registered	Bastion authorized_keys file absent	Bastion rejects SSH; logged as "unknown device"
HMAC secret file missing	phonehome_init() returns -1	Phone-home service disabled; other DoIP services unaffected
Provisioning token reuse attempt	JTI found in used-token DB	Registration API returns 401

---

11. Testing Procedures

11.1 Unit Tests

UT-01: HMAC Validation — Valid Input **Target:** `phonehome_routine_handler()`
Method: Call handler with a correctly formed DoIP PDU including valid HMAC.
Expected: Returns positive response `{0x71, 0x01, 0xF0, 0xA0, 0x02}`

```
void test_hmac_valid(void) {
    uint8_t nonce[8] = {0x01,0x02,0x03,0x04,0x05,0x06,0x07,0x08};
    uint8_t hmac[32];
    unsigned int hmac_len = 32;
    HMAC(EVP_sha256(), test_secret, 32, nonce, 8, hmac, &hmac_len);

    uint8_t pdu[4 + 8 + 32] = {0x31, 0x01, 0xF0, 0xA0};
    memcpy(pdu + 4, nonce, 8);
    memcpy(pdu + 12, hmac, 32);

    uds_response_t r = phonehome_routine_handler(pdu, sizeof(pdu));
    assert(r.data[0] == 0x71);
    assert(r.data[4] == 0x02);  // routineRunning
}
```

UT-02: HMAC Validation — Invalid HMAC **Expected:** NRC response `{0x7F, 0x31, 0x35}`

UT-03: Replay Protection — Same Nonce Twice **Method:** Call handler twice with identical nonce and valid HMAC.
Expected: First call returns positive response; second call returns NRC 0x24.

UT-04: Replay Cache Expiry **Method:** Insert nonce with expiry set to `now - 1`. Call handler with same nonce.
Expected: Returns positive response (nonce treated as new).

UT-05: Short PDU **Method:** Call handler with PDU shorter than minimum (< 44 bytes).
Expected: NRC 0x13 (incorrectMessageLength).

UT-06: Key Generation Entropy Check **Method:** Set `/proc/sys/kernel/random/entropy_avail` mock to 100. Run `phonehome-keygen.sh`.
Expected: Script exits with non-zero status; no key file created; error logged.

UT-07: Key Permission Enforcement **Method:** Run `phonehome-keygen.sh` successfully.
Expected: `stat -c %a /etc/phonehome/id_ed25519` returns `600`.

UT-08: Registration — Valid Public Key **Method:** POST to `/api/v1/devices/register` with valid ED25519 public key and valid provisioning token.
Expected: HTTP 201; device appears in database; authorized_keys file created.

UT-09: Registration — Invalid Key Format **Method:** POST with RSA key instead of ED25519.
Expected: HTTP 400.

UT-10: Registration — Replayed Provisioning Token **Method:** POST twice with same JWT.
Expected: Second call returns HTTP 401.

11.2 Integration Tests

IT-01: End-to-End DoIP Trigger on Loopback **Setup:** DCU DoIP server running on test VM. FC-1 simulator sends DoIP RoutineControl 0x31/0xF0A0 with valid HMAC.
Expected: 1. DoIP server responds positively within 200ms 2. `phonehome-connect.sh` process is spawned (verify with `ps`) 3. Lock file `/var/run/phonehome.lock` exists

IT-02: DoIP Trigger → SSH Tunnel (Lab Bastion) **Setup:** Lab Bastion running. DCU registered. FC-1 simulator sends trigger.
Expected: 1. DCU spawns SSH process 2. Bastion logs show `phonehome-<DCU_SERIAL>` user connecting 3. Reverse tunnel port assigned and reachable from Bastion

IT-03: FC-1 Signal Relay **Setup:** FC-1 hardware with SSH tunnel active to Lab Bastion. Dispatcher sends wake command via FC-1 tunnel.
Expected: `phonehome-relay.sh` executes on FC-1; DoIP trigger arrives at DCU within 2 seconds.

IT-04: Full Operator Flow **Setup:** Complete stack (Web UI → Backend → Bastion → FC-1 → DCU).
Steps: 1. Operator clicks Connect button 2. Web UI polls `/api/v1/phonehome/status/DCU-XXX` 3. Tunnel becomes ready 4. Operator SSHs to Bastion on returned port
Expected: Operator reaches DCU SSH prompt within 10 seconds of clicking.

IT-05: Concurrent Tunnel Prevention **Setup:** Active tunnel exists on DCU.
Method: Send second DoIP trigger.
Expected: NRC 0x21 (busyRepeatRequest); existing tunnel unaffected.

IT-06: Network Interruption Recovery **Setup:** Tunnel established. 4G connection simulated-dropped for 90 seconds.
Expected: `ServerAliveCountMax=3` causes SSH to exit after ~90s; lock file removed; DCU is ready for a new trigger.

11.3 End-to-End Tests

E2E-01: Provisioning from Factory State **Method:** Start with a blank DCU (no keys, no sentinel files). Power on.
Expected: 1. `phonehome-keygen.service` runs and generates keys 2. `phonehome-register.service` retries until network available 3. Registration succeeds; sentinel written; provisioning token deleted 4. Phone-home trigger works within 5 minutes of first boot

E2E-02: Fleet Registration Stress Test **Method:** Simultaneously provision 50 DCU VMs against the Registration API.
Expected: All register successfully within 5 minutes; no duplicate entries; all authorized_keys files created correctly.

E2E-03: 24-Hour Tunnel Stability **Method:** Establish DCU reverse tunnel. Leave open for 24 hours with `ServerAliveInterval=30`.
Expected: Tunnel remains active throughout; `phone_home_log` shows single connected event with no disconnect.

11.4 Security Tests

ST-01: HMAC Brute Force Resistance **Method:** Send 1000 DoIP requests with random HMAC values.
Expected: All rejected with NRC 0x35; no false positives; no server crash; security events logged for each.

ST-02: Nonce Replay After Cache Eviction **Method:** Fill replay cache with 64 entries. Submit the 65th (oldest evicted nonce) again.
Expected: Accepted (eviction is expected behavior). Document this as a known design trade-off — the cache size of 64 with 300s TTL means nonces can be replayed after ~64 * (arrival_interval).

For the use case of one trigger per session, this is acceptable. If higher frequency is expected, increase `REPLAY_CACHE_SIZE`.

ST-03: Man-in-the-Middle on DoIP LAN **Method:** ARP-spoof the DCU on the local LAN from a test device. Send forged DoIP trigger without HMAC secret.
Expected: NRC 0x35; no tunnel initiated.

ST-04: Bastion Host Key Substitution **Method:** Start an SSH server on a different host with a different host key. Modify DNS to point `bastion-dev.imatrixsys.com` at it. Trigger phone-home.
Expected: SSH exits with host key verification failure; no connection to rogue server; security event logged.

ST-05: Private Key Extraction Attempt **Method:** As a non-root user on the DCU, attempt to read `/etc/phonehome/id_ed25519`.
Expected: Permission denied (0600 + root ownership).

ST-06: Provisioning Token Reuse **Method:** Attempt to use a provisioning JWT for a second registration of a different device.
Expected: Bastion returns HTTP 401 (token serial mismatch).

11.5 Fleet Regression Test

Run after any change to phone-home components before release:

```
#!/bin/sh
# fleet-regression.sh
# Runs against a fleet simulation environment with N virtual devices

N=10
PASS=0
FAIL=0

for i in $(seq 1 $N); do
    SERIAL="TEST-DCU-$(printf '%06d' $i)"
    RESULT=$(curl -s -o /dev/null -w "%{http_code}" \
        -X POST https://bastion-test.example.com/api/v1/phonehome/wake \
        -H "Authorization: Bearer $OPERATOR_TOKEN" \
        -d "{\"dcu_serial\":\"$SERIAL\"}")
    if [ "$RESULT" = "202" ]; then
        PASS=$((PASS+1))
    else
        FAIL=$((FAIL+1))
        echo "FAIL: $SERIAL returned HTTP $RESULT"
    fi
done
```

```
echo "Results: $PASS passed, $FAIL failed out of $N"
[ $FAIL -eq 0 ] || exit 1
```

---

12. Recommended Source Files and Dependencies

The following existing projects provide tested, production-quality code that you should incorporate rather than re-implement.

12.1 DoIP Client Utility (for FC-1 → DCU signaling)

Option	Repo	Notes
doip-client (Python)	`https://github.com/doip/doip-client`	Pure Python; works on BusyBox with Python 3
automotive-tools	`https://github.com/pylessard/python-udsoncan`	Python UDS library; pair with `python-doip`
Open-source ISO 13400 C impl	Write minimal custom client	~200 lines of C; easiest for embedded; see template below

Minimal DoIP raw client for BusyBox (no Python required):

```c
// doip_send.c - minimal DoIP RoutineControl sender
// Compile: gcc -o doip-client doip_send.c
// Usage: doip-client <host> <port> <src_addr_hex> <tgt_addr_hex> <payload_hex>

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <netdb.h>
#include <unistd.h>

// DoIP Generic Header: version(1) + inv_version(1) + payload_type(2) + payload_length(4)
// DoIP Diagnostic Message: source_addr(2) + target_addr(2) + user_data(N)
// Payload type 0x8001 = Diagnostic Message

static void hex2bin(const char *hex, uint8_t *bin, size_t *len) {
    *len = 0;
    while (*hex && *(hex+1)) {
        sscanf(hex, "%2hhx", &bin[(*len)++]);
        hex += 2;
    }
}

int main(int argc, char **argv) {
```

```
    if (argc < 6) {
        fprintf(stderr, "Usage: %s <host> <port> <src_hex> <tgt_hex> <payload_hex>\n", argv[0]
        return 1;
    }
    uint16_t src = (uint16_t)strtol(argv[3], NULL, 16);
    uint16_t tgt = (uint16_t)strtol(argv[4], NULL, 16);
    uint8_t payload[512];
    size_t plen;
    hex2bin(argv[5], payload, &plen);

    uint32_t diag_len = 4 + plen;  // src(2) + tgt(2) + data
    uint8_t frame[8 + 4 + 512];
    frame[0] = 0x02;  frame[1] = 0xFD;  // version + inverse
    frame[2] = 0x80;  frame[3] = 0x01;  // payload type: Diagnostic Message
    frame[4] = (diag_len >> 24) & 0xFF;
    frame[5] = (diag_len >> 16) & 0xFF;
    frame[6] = (diag_len >> 8)  & 0xFF;
    frame[7] =  diag_len        & 0xFF;
    frame[8]  = (src >> 8) & 0xFF;  frame[9]  = src & 0xFF;
    frame[10] = (tgt >> 8) & 0xFF;  frame[11] = tgt & 0xFF;
    memcpy(frame + 12, payload, plen);

    struct addrinfo hints = {0}, *res;
    hints.ai_family = AF_INET;
    hints.ai_socktype = SOCK_STREAM;
    if (getaddrinfo(argv[1], argv[2], &hints, &res) != 0) { perror("getaddrinfo"); return 1; }

    int fd = socket(res->ai_family, res->ai_socktype, 0);
    if (connect(fd, res->ai_addr, res->ai_addrlen) != 0) { perror("connect"); return 1; }
    write(fd, frame, 12 + plen);
    // Read response (simplified - in production, parse DoIP response fully)
    uint8_t resp[64];
    int n = read(fd, resp, sizeof(resp));
    close(fd);
    freeaddrinfo(res);
    if (n > 8 && resp[8] == 0x71) { printf("OK: positive response\n"); return 0; }
    if (n > 10 && resp[8] == 0x7F) { printf("NRC: 0x%02x\n", resp[10]); return 1; }
    return 1;
}
```

12.2 OpenSSH (BusyBox Dropbear alternative)

The DCU phone-home script requires SSH client features (`-R` for reverse tunnel, `ServerAliveInterval`, `BatchMode`). Dropbear supports all of these but verify your build includes reverse tunnel support (`DROPBEAR_REVERSE_TUNNEL` compile flag). Standard OpenSSH `ssh` is preferred if the DCU filesystem budget allows it.

12.3 OpenSSL / LibreSSL

Required for HMAC-SHA256 in the C handler. Available on BusyBox systems via `libcrypto`. Alternative: use `hmac` from BusyBox's `libbb` if OpenSSL is not available, but verify the API matches.

12.4 Key Source Files to Create or Obtain

File	Source / Action
`/usr/sbin/phonehome-keygen.sh`	Create per Section 5.1
`/usr/sbin/phonehome-register.sh`	Create per Section 5.2
`/usr/sbin/phonehome-connect.sh`	Create per Section 5.3
`phonehome_handler.c` / `phonehome_handler.h`	Create per Section 5.4 / 6.2
`/usr/sbin/phonehome-relay.sh`	Create per Section 5.5 (FC-1)
`doip_send.c` (doip-client binary)	Create per Section 12.1 (FC-1)
`registration_api.py`	Create per Section 5.6 (Bastion)
`signal_dispatcher.py`	Create per Section 5.7 (Bastion)
`/etc/systemd/system/phonehome-keygen.service`	Create per Section 5.1
`/etc/systemd/system/phonehome-register.service`	Create per Section 5.2
`/var/lib/phonehome/schema.sql`	Create per Section 5.6
`/etc/ssh/sshd_config` (additions)	Modify per Section 9.3
`fleet-regression.sh`	Create per Section 11.5

12.5 Existing Codebase Review Checklist

Before integrating, review these existing files and APIs in your DoIP server codebase:

- ☐ UDS dispatcher / handler registration function — identify how to register SID 0x31 + routineIdentifier 0xF0A0
- ☐ Existing RoutineControl handlers — ensure 0xF0A0 does not conflict with any existing routineIdentifier
- ☐ Fork/process management policy — confirm the server design permits `fork()` from within a handler
- ☐ Thread safety model — determine whether the replay cache mutex implementation is needed
- ☐ Logging infrastructure — integrate with existing `syslog` or proprietary logging framework
- ☐ Build system (Makefile/CMake) — add `phonehome_handler.c` and `-lssl -lcrypto` linker flags
- ☐ NRC constants — verify 0x35 and 0x24 are not reassigned in your UDS implementation

---

13. Open Issues and Decision Log

ID	Issue	Decision	Date
DEC-01	Key origin: FC-1 generates DCU keys vs DCU self-generates	**DCU self-generates. Private key never leaves DCU.** ISO 21434 compliance.	—
DEC-02	DoIP channel for key delivery	**Abandoned.** DoIP used only for phone-home trigger, not key delivery.	—
DEC-03	SSH algorithm selection	**ED25519 selected.** Compact, fast on embedded, 128-bit equivalent security.	—
DEC-04	Reverse tunnel port allocation	**Server auto-assign (port 0).** Dispatcher monitors sshd for assigned port. Implementation of tunnel-monitor daemon is outstanding.	OPEN
DEC-05	Replay cache size vs. nonce window	**64 entries, 300s TTL.** Appropriate for one trigger per session. Revisit if multi-trigger per minute use case arises.	—
DEC-06	Dropbear vs. OpenSSH on DCU	**TBD.** Depends on DCU firmware size budget. Dropbear is ~400KB; OpenSSH is ~2MB. Verify Dropbear reverse tunnel support in your build.	OPEN
DEC-07	Tunnel-monitor daemon implementation	Outstanding. Must detect assigned reverse tunnel port from sshd. Options: parse `/var/log/auth.log`, use `ss -tlnp`, or use `ControlSocket` SSH multiplexer.	OPEN

ID	Issue	Decision	Date
DEC-08	HMAC secret delivery to DCU at manufacture	**NVM / EOL programming.** Exact mechanism is manufacture-toolchain-specific.	OPEN